

Orientación a Objetos II

2026

Explicación de práctica
Semana del 4 de mayo



FACULTAD DE INFORMATICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Ejercicios de la semana

- Ejercicio 19 - File Manager
- Ejercicio 20 - Acceso a la base de datos
- Ejercicio 21 - Estación meteorológica
- Ejercicio 22 - Filtros de Imágenes

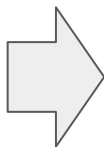
Ejercicio 22 - Filtros de Imágenes

En este proyecto encontrará la librería **ImageFilter** que implementa algunos filtros básicos sobre imágenes PNG tales como: **Rainbow**, **Repeater**, **RGBShifter**, entre otros.

La librería incluye una herramienta (**PNGFilterLauncher**) que permite aplicar secuencia de filtros sobre una archivo de entrada y generar un archivo (.png) de salida.



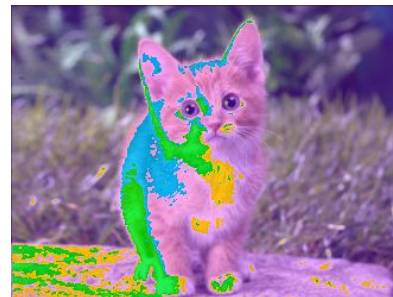
PNG original



Rainbow



Repeater



RGBShifter

Ejercicio 22 - Estructura del proyecto



Dentro del directorio **ImageFilters** están los diferentes filtros disponibles que se pueden usar para transformar las imágenes.

PNGFilterLauncher es una clase con un método *main* que recibe los **argumentos** de ejecución. Esto permite indicar la imagen sobre la que se va a trabajar, qué filtros aplicar y en qué orden

Ejercicio 22 - Estructura del proyecto

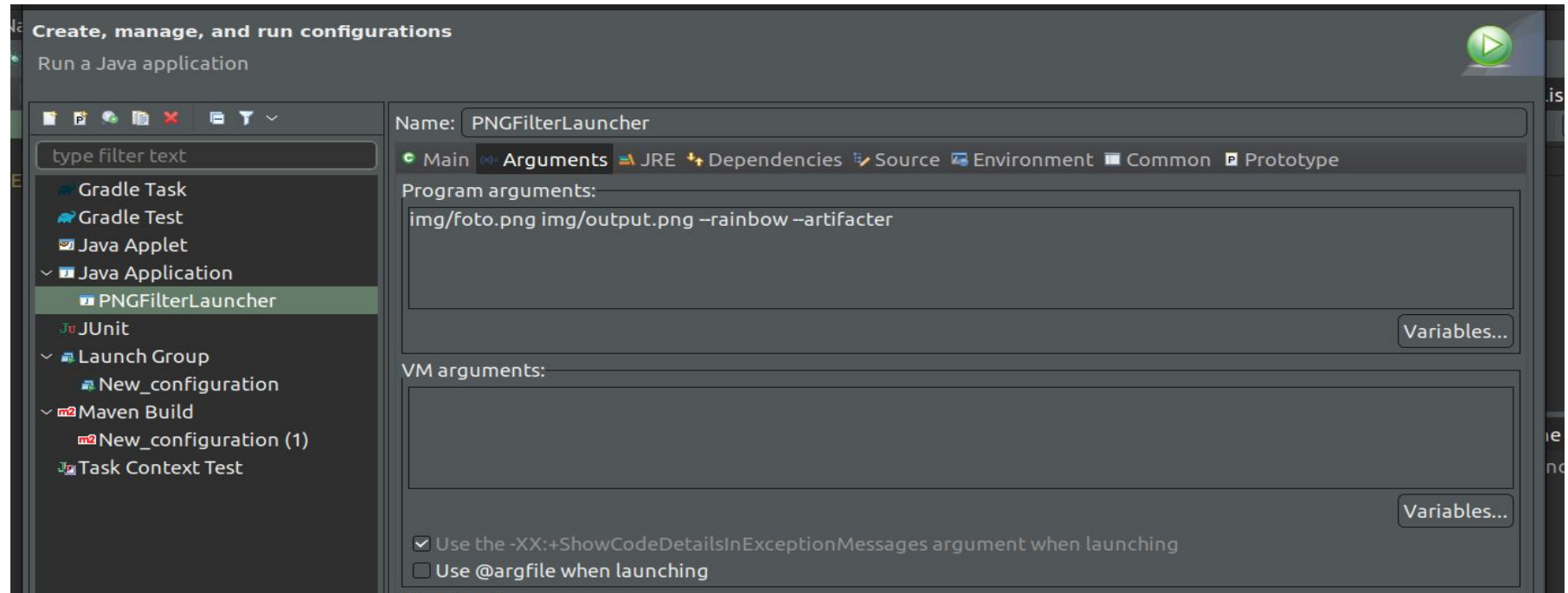
```
14 public class PNGFilterLauncher {
15     private static Map<String, Filter> filters = new HashMap<String, Filter>();
16     private static ArrayList<Filter> activeFilters = new ArrayList<Filter>();
17     private static File inputFile;
18     private static File outputFile;
19
20     Run | Debug
21     public static void main(String args[]) {
22         // Initialize the list of filters
23         PNGFilterLauncher.initializeFilters();
24
25         // Process the arguments
26         if (args.length < 2) {
27             System.out.println(x:"Usage: java Main input-file output-file [--filter]");
28             return;
29         }
30         PNGFilterLauncher.processArgs(args);
31     }
32 }
```

Dentro del directorio **ImageFilters** están los diferentes filtros disponibles que se pueden usar para transformar las imágenes.

PNGFilterLauncher es una clase con un método *main* que recibe los **argumentos** de ejecución. Esto permite indicar la imagen sobre la que se va a trabajar, qué filtros aplicar y en qué orden

Ejercicio 22 - Parámetros de entrada

Para configurar los argumentos que recibe la aplicación, buscar en el menú principal de Eclipse: **“Run” > “Run configurations”** e indicarlos en **“arguments”**



Ejercicio 22 - Parámetros de entrada

- En **Windows**

```
img\foto.png img\output.png --rainbow --artifacter
```

- En

Linux/Mac:

```
img/foto.png img/output.png --rainbow --artifacter
```

- **img/foto.png**: PNG original. Puede ser cualquier imagen PNG, el proyecto trae una de ejemplo.
- **img/output.png**: Ruta en donde queremos que deje el resultado.
- **Lista de filtros a aplicar**: Explorar el código para ver qué otras opciones existen

Forma de ejecutar alternativa

Ubicado en el directorio del proyecto, y luego de su compilación con Maven, ejecutar en consola:

```
java -cp target/classes ar.edu.info.oo2.filtros.PNGFilterLauncher  
img/foto.png img/salida.png --rainbow
```

También se pueden usar maneras propias que provea el IDE

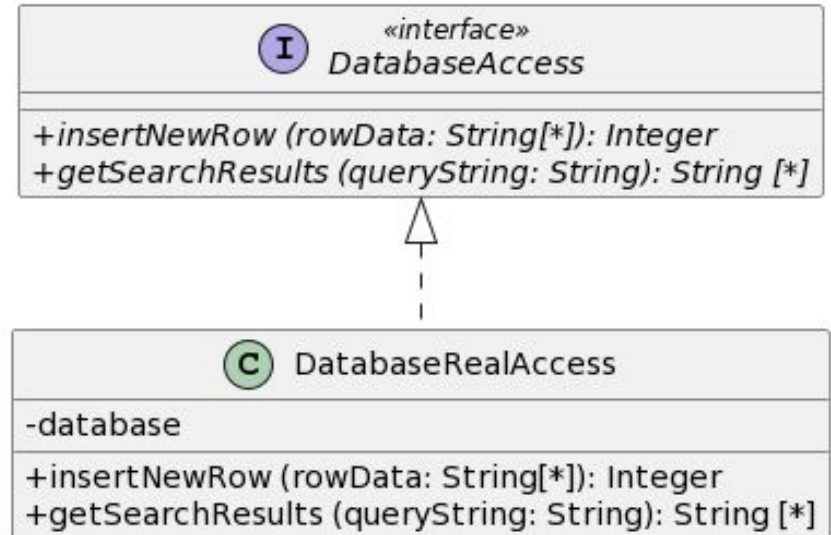
Ejercicio 22 - Tareas

1. Descargue el proyecto y pruebe los diferentes Filtros
2. Analice el código y documente el proyecto con un **Diagrama de Clases**
3. Evalúe cuál de los siguientes patrones mejor describe el diseño de los Filtros: **Template Method**, **Strategy**, **Decorator**. Para realizar la evaluación se sugiere contestar las siguientes preguntas aplicadas a cada uno de los patrones
4. Se requiere crear el filtro **Monochrome** y se presenta un pseudocódigo cómo guía

Ejercicio 20 - Acceso a la base de datos

Queremos acceder a una base de datos que contiene información sobre cómics.

Este acceso está dado por el comportamiento de la clase **DatabaseRealAccess** con el siguiente protocolo y modelado como muestra la siguiente figura.



Ejercicio 20 - Acceso a la base de datos

En este caso, ustedes recibirán una implementación prototípica de la clase **DatabaseRealAccess** que simula el uso de una base de datos de la siguiente forma:

```
1  // Instancia una base de datos que posee dos filas
2  database = new DatabaseRealAccess();
3
4  // Retorna el siguiente arreglo: ['Spiderman', 'Marvel'].
5  database.getSearchResults("select * from comics where id=1");
6
7  // Retorna 3, que es el id que se le asigna
8  database.insertNewRow(Arrays.asList("Patoruzú", "La flor"));
9
10 // Retorna el siguiente arreglo: ['Patoruzú', 'La flor'], ya que lo insertó antes
11 database.getSearchResults("select * from comics where id=3");
```

Ejercicio 20 - Acceso a la base de datos

En esta oportunidad, usted debe proveer una solución utilizando un patrón que le permita brindar protección al acceso a la base de datos de forma que lo puedan realizar solamente usuarios que se hayan autenticado previamente.



Ejercicio 20 - Comportamiento esperado

```
1 // Una contraseña muy difícil.
2 String password = "p455w0rD"
3
4 // Instancia una base de datos que posee dos filas, y se accede con contraseña
5 database = new YourClass(... some mysterious parameters ..., password);
6
7 // No retorna nada.
8 database.getSearchResults("select * from comics where id=1");
9
10 // Pero... si previamente realizó el login con la contraseña correcta...
11 database.login(password)
12 // Retorna el siguiente arreglo: ['Spiderman' 'Marvel'].
13 database.getSearchResults("select * from comics where id=1");
```

- Enfocarse en la protección de la base de datos
- La “implementación prototípica” sirve para poder “probar”
- Test provistos por la cátedra

Ejercicio 20 - Acceso a la base de datos



Hay que proveer una solución utilizando un patrón que le permita **brindar protección al acceso a la base de datos** de forma que lo puedan realizar solamente usuarios que se hayan autenticado previamente.

Patrón Proxy

Propósito: proporcionar un intermediario de un objeto para controlar su acceso.

Aplicabilidad: cuando se necesita una referencia más flexible hacia un objeto:

- ✓ **Virtual proxy:** demorar la construcción de un objeto hasta que sea realmente necesario.
- ✓ **Protection proxy:** Restringir el acceso a un objeto por seguridad.
- ✓ **Remote proxy:** representar un objeto remoto en el espacio de memoria local

Patrón Proxy - Participantes

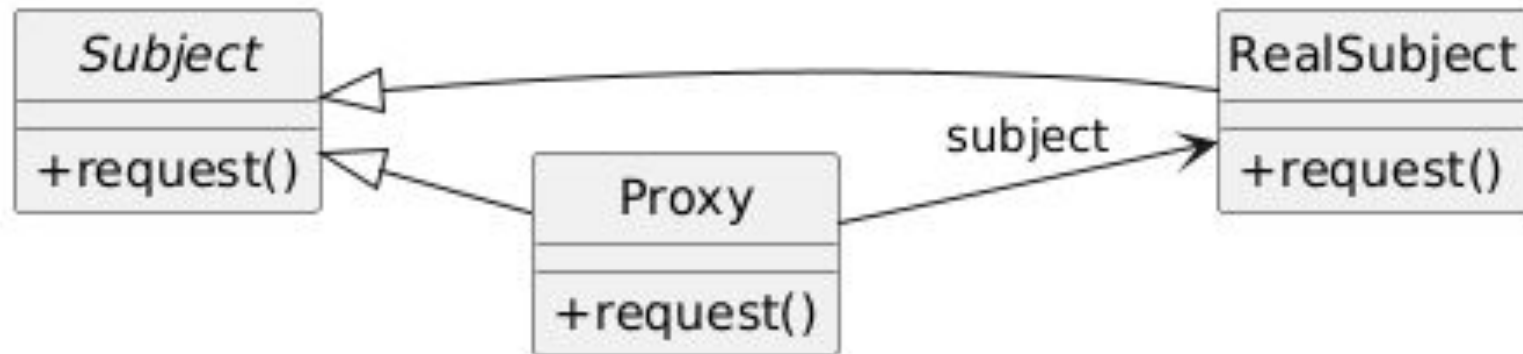
- **Subject:** Define la interfaz para el **Real Subject** y el **Proxy**, de modo que el **Proxy** pueda ser utilizado en cualquier lugar donde se espere un **Real Subject**
- **Real Subject:** Define el objeto real que el **Proxy** representa
- **Proxy:**
 - Mantiene una referencia que le permite acceder a un **Real Subject**
 - Controla el acceso y puede ser responsable de su creación o destrucción
 - **Diferentes responsabilidades de acuerdo al tipo de proxy**

Patrón Proxy - Participantes (2)

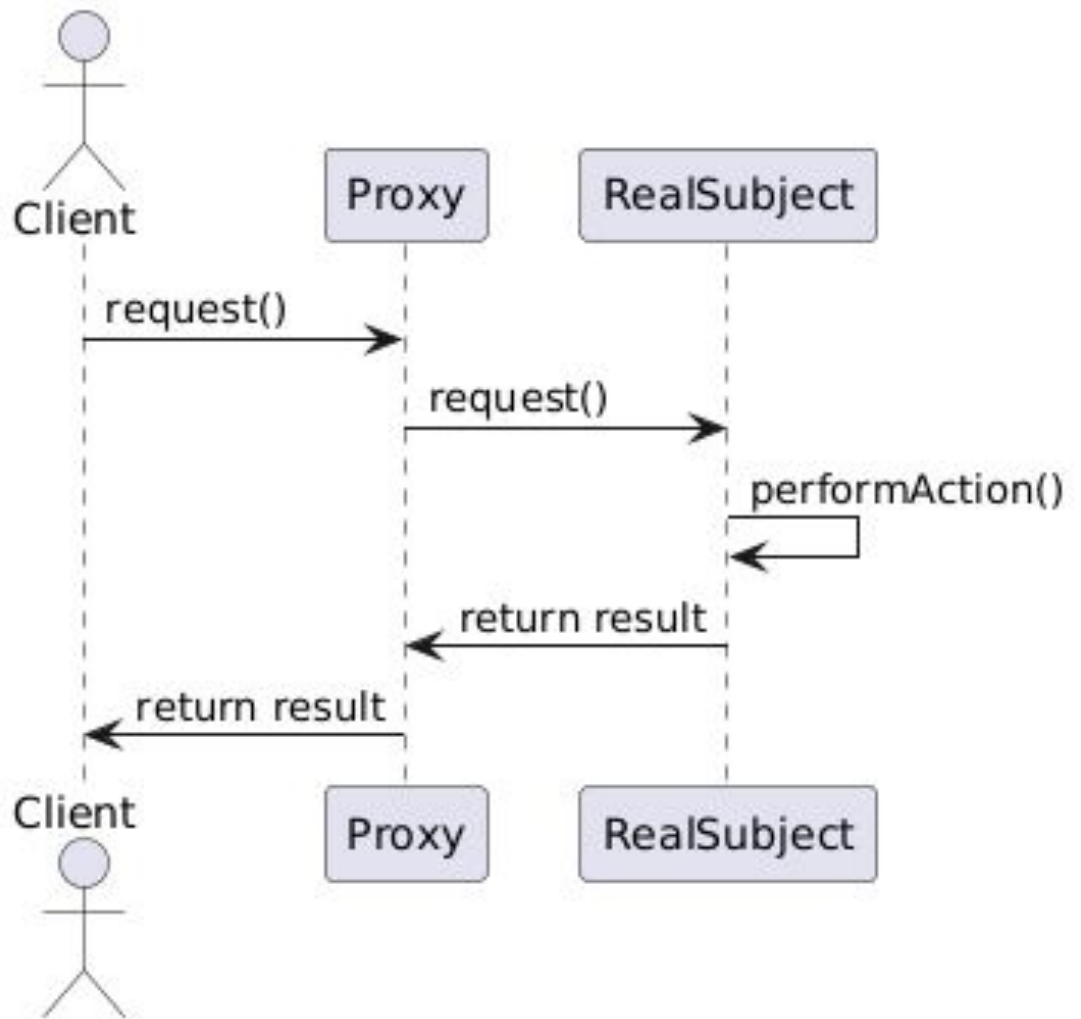
- **Proxy:**

- **Virtual Proxy:** crean objetos costosos bajo demanda (lazy loading)
- **Protection Proxy:** controlan los derechos/permisos de acceso al objeto real
- **Remote Proxy:** Envían solicitudes a través de una red. Es decir, es un objeto que no está en el espacio de memoria local. Se ocupan de comunicarse con un objeto distribuidos por la red, y de serializar/deserializar mensajes y respuestas

Patrón Proxy - Estructura



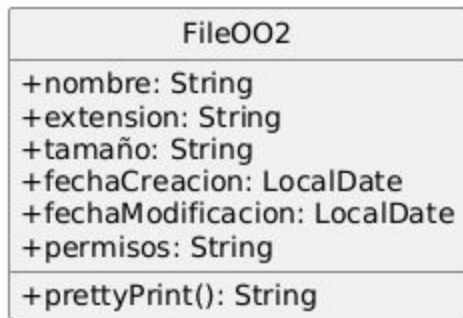
El **Proxy** actúa como intermediario entre el cliente y el **RealSubject**, enviando las solicitudes al **RealSubject** y luego devolviendo los resultados al cliente.



Patrón Proxy - Consecuencias

- + El cliente no debería notar si está usando el **Real Subject** o el **Proxy**
- + Indirección en el acceso al objeto -> Permite controlar el acceso al objeto real, lo que ofrece flexibilidad en su uso
- Introduce complejidad
- Dificulta depuración y trazabilidad si el **Proxy** hace muchas operaciones intermedias

Ejercicio 19 - File Manager



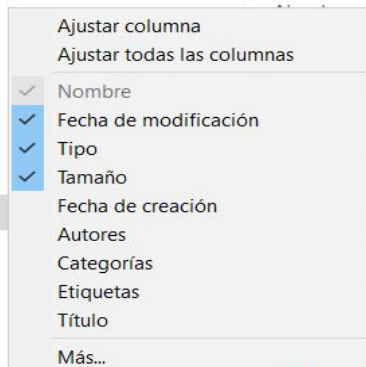
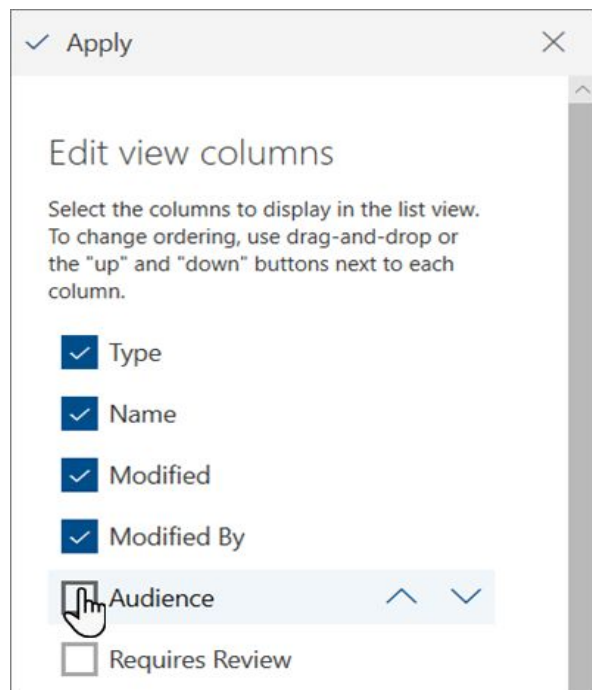
prettyPrint()
retorna el nombre
del archivo

En el File Manager el usuario debe poder elegir cómo se muestra un archivo (instancia de la clase FileOO2), es decir, cuáles de los aspectos mencionados anteriormente se muestran, y en qué orden. Esto quiere decir que un usuario podría querer ver los archivos de muchas maneras. Algunas de ellas son:

- nombre - extensión
- nombre - fecha de creación - extensión
- nombre - tamaño - permisos - extensión

Son sólo **algunos** ejemplos

Ejercicio 19 - File Manager (2)



	Fecha de modificación	Tipo
ins-grammaticales-5e-année-exer...	11/9/2021 14:39	Adobe Acrobat
Poucette - Essai complet.pdf	18/6/2021 09:38	Adobe Acrobat
s.pdf	2/7/2021 13:00	Adobe Acrobat
s.docx	2/7/2021 13:00	Documento de
memo_gr_neb21_U5_articulateurs...	3/12/2020 14:00	Adobe Acrobat
gconnect.pdf	2/7/2021 11:55	Adobe Acrobat
31_B2 - les liaisons.pdf	19/8/2021 10:53	Adobe Acrobat
de révision.pdf	25/8/2021 20:13	Adobe Acrobat
obante_.pdf	20/9/2021 16:28	Adobe Acrobat
at_B22.pdf	22/9/2021 18:47	Adobe Acrobat
20220414 Los Gigar	22/9/2021 18:43	Adobe Acrobat
auto - AF164IP	19/9/2021 09:24	Documento de
OneDrive		
Este equipo		
Red		
B22-IFM_Certificat B22.pdf	18/9/2021 19:01	Adobe Acrobat
B22 Bilan final.docx	19/9/2021 10:03	Adobe Acrobat
B2.2 Petit Quiz final _ relecture de tentati...	31/3/2022 11:12	Carpeta de arc
B2.2 Evaluation finale _ relecture de tenta...	31/3/2022 11:12	Carpeta de arc
Unite 5	31/3/2022 11:12	Carpeta de arc
Unite 4	31/3/2022 11:12	Carpeta de arc
Unite 3	31/3/2022 11:12	Carpeta de arc
Unite 2	31/3/2022 11:12	Carpeta de arc
Unite 1	31/3/2022 11:12	Carpeta de arc
Unite 0	31/3/2022 11:12	Carpeta de arc

Se podrían generar todas las combinaciones posibles.

Ejercicio 19 - File Manager (3)

Es decir, un objeto cliente, por ejemplo, el FileManager, le enviará al FileOO2 el mensaje `prettyPrint()`.

De acuerdo a cómo el usuario lo haya configurado se deberá retornar un String con los aspectos seleccionados por el usuario en el orden especificado por éste.

Considere que un mismo archivo podría verse de formas diferentes desde distintos puntos del sistema, y que el usuario podría cambiar la configuración del sistema (qué y en qué orden quiere ver) en runtime.

Patrón Decorator

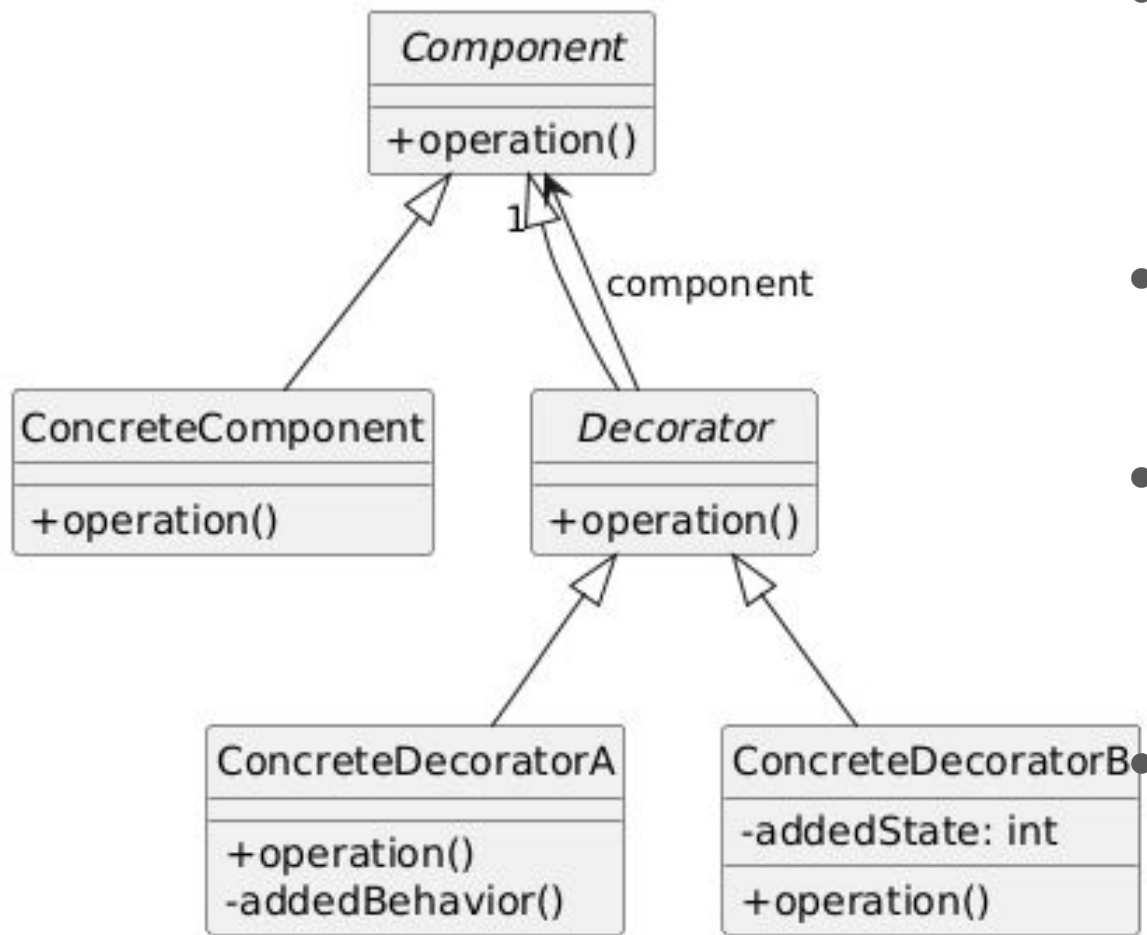
Propósito: Agregar comportamiento a un objeto dinámicamente y en forma transparente.

Aplicabilidad: Use el patrón Decorator cuando:

- ✓ agregar responsabilidades a objetos individualmente y en forma transparente (sin afectar otros objetos)
- ✓ quitar responsabilidades dinámicamente
- ✓ cuando subclasificar es impráctico

Patrón Decorador - Participantes

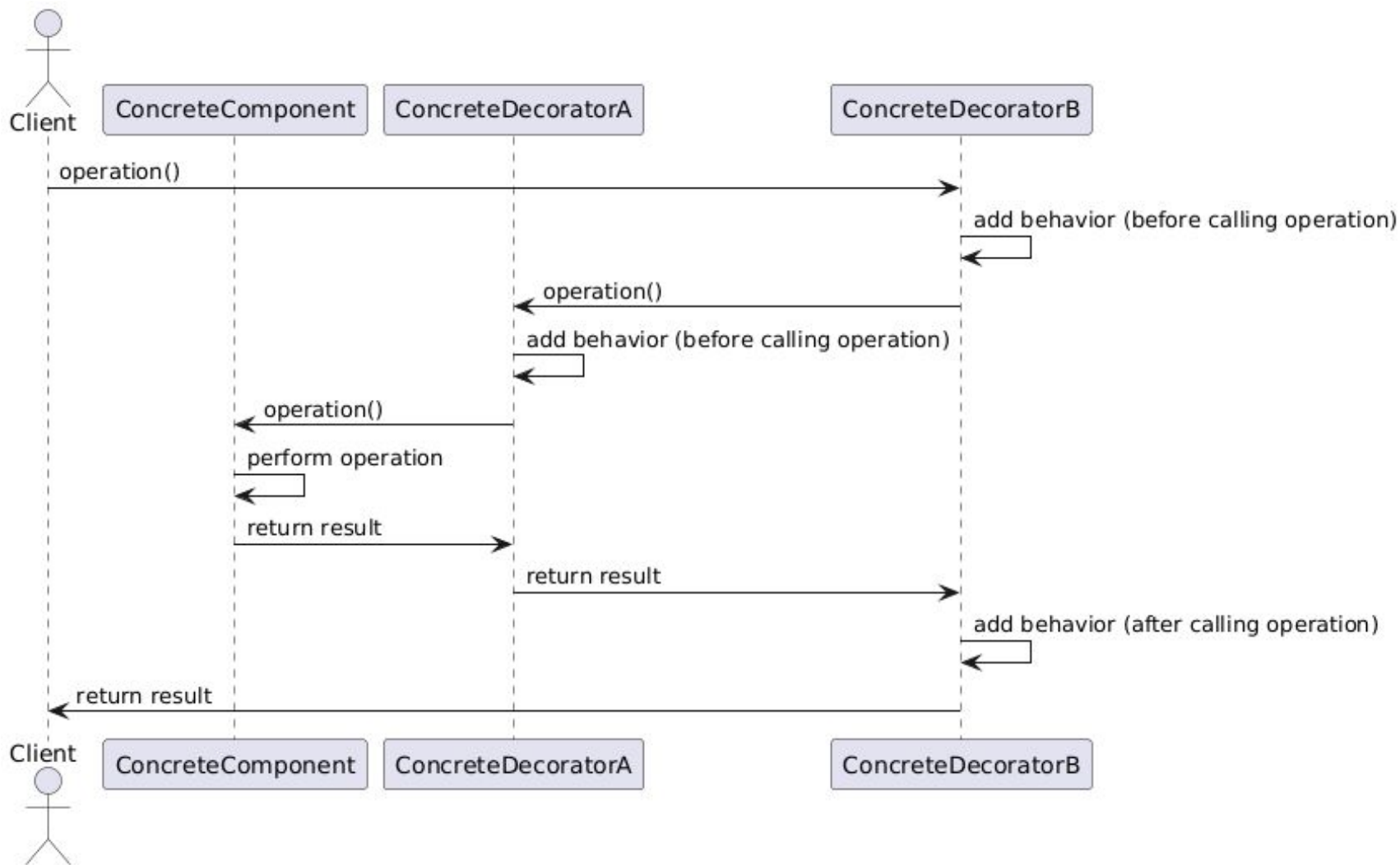
- **Component:** Define la interfaz para los objetos a los que se les pueden añadir responsabilidades de manera dinámica
- **Concrete Component:** Define un objeto al cual se le pueden agregar responsabilidades adicionales.
- **Decorator:** Mantiene una referencia a un objeto **Component** y define una interfaz que se ajusta a la del **Component**.
- **Concrete Decorator:** agrega responsabilidades al **Component**

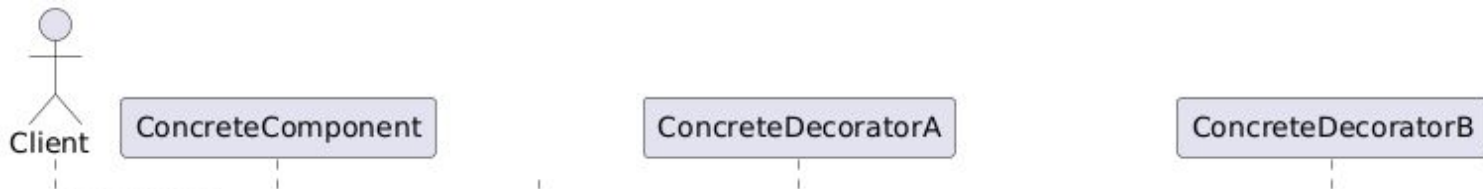


- **Component:** Interfaz base de los objetos que pueden ser decorados.
- **Concrete Component:** Objeto real que recibe decoraciones.
- **Decorator:** Clase abstracta que envuelve y delega en un **Component**
- **Concrete Decorator:** Implementa nuevas responsabilidades

Patrón Decorator - Consecuencias

- + Permite mayor flexibilidad que la herencia.
- + Permite agregar funcionalidad incrementalmente.
- Mayor cantidad de objetos.
- Mayor dificultad para depurar.





```
1 component = new ConcreteDecoratorB(  
2     new ConcreteDecoratorA(  
3         new ConcreteComponent()  
4     )  
5 );  
6  
7 component.operation();
```

```
1 Component component = new ConcreteComponent();  
2  
3 component = new ConcreteDecoratorA(component);  
4 component = new ConcreteDecoratorB(component);  
5  
6 component.operation();
```



Patrón Decorator - Cuestiones de implementación

- Misma interface entre componente y decorador.
- No hay necesidad de la clase Decorator abstracta, si se tiene un solo decorador.
- Cambiar el “skin” vs cambiar sus “guts”
- Decorador preserva la interface del objeto para el cliente
- Decoradores pueden y suelen anidarse

